PRESENTASI TUGAS KELOMPOK

Analisis & Implementasi Perbandingan

Layered vs Non-Layered Web Application Architecture

Studi Kasus: Customer Management System (PHP Native, REST API JSON, & MySQL)

1. Pendahuluan & Konsep

- Pengertian Non-Layered & 3-Layer
- secara Netral & Objektif

2. Diagram & Kode Program

- Pemisahan File/Folder, Hubungan
- HTML/CSS/PHP/SQL, & Bukti Kode

3. Demo Web & Alur Proses

- Tampilan CRUD, Validasi Email,
- & Alur Request-Response

4. Tabel Pengujian & Diskusi

- 1 Tabel Utama Hasil Uji Coba,
- Siapa Lebih Unggul, & Alasannya

Pendahuluan: Latar Belakang & Tujuan Perbandingan

Mengenal Dua Pendekatan Arsitektur Aplikasi Web yang Akan Diimplementasikan & Diuji

Dua Pendekatan Arsitektur Aplikasi Web

1. Pendekatan Pertama: Non-Layered Architecture

Aplikasi dikembangkan tanpa pemisahan layer secara tegas. Tampilan, proses bisnis, dan akses database dapat berada dalam satu file atau beberapa file yang belum memiliki pemisahan tanggung jawab yang jelas.

2. Pendekatan Kedua: 3-Layer Architecture

Aplikasi dikembangkan dengan memisahkan tanggung jawab secara tegas menjadi tiga lapisan: Presentation Layer, Business Logic Layer, dan Data Layer.

3. Mengapa Perlu Dianalisis & Dibandingkan?

Kedua pendekatan memiliki cara kerja dan karakteristik masing-masing. Oleh karena itu, keduanya diimplementasikan pada studi kasus yang sama untuk diuji secara objektif.

Tujuan & Metodologi Perbandingan

1. Membangun 2 Aplikasi dengan Fitur Identik

Membangun Customer Management System (6 fitur wajib) menggunakan pendekatan Non-Layered (1 file PHP) dan 3-Layer (3 folder + REST API JSON).

2. Membandingkan Struktur File/Folder & Kode

Menunjukkan perbedaan hubungan HTML, CSS, JS, PHP, dan SQL melalui diagram arsitektur serta screenshot kode kedua pendekatan.

3. Menguji & Menilai pada Tabel Perbandingan Akhir

Menguji kedua aplikasi pada 8 unsur (kecepatan buat awal, individu vs tim, penelusuran bug, respons UI, dll.) untuk melihat keunggulan masing-masing secara objektif.

Konsep Pendekatan 1: Non-Layered Architecture

Pengertian, Karakteristik, & Cara Kerja Aplikasi Tanpa 3-Layer (Sesuai Halaman 14–16 PDF)

Pengertian Non-Layered Architecture (Hal. 14 PDF)

1. Definisi Tanpa Pemisahan Layer (Hal. 14 & 18 PDF)

Aplikasi dikembangkan tanpa pemisahan layer secara tegas. Tampilan, proses bisnis, dan akses database dapat berada dalam satu file atau beberapa file yang belum memiliki pemisahan tanggung jawab yang jelas.

2. Implementasi Studi Kasus (1 File index.php — Hal. 14 & 20)

Pada studi kasus ini, seluruh proses ditempatkan dalam 1 file PHP (index.php): menampilkan form, menerima input, memvalidasi data, mengakses MySQL, hingga menampilkan hasil.

3. Catatan Penting (Halaman 16 PDF)

Aplikasi tanpa 3-layer ini bukan berarti tidak memiliki tiga tanggung jawab tersebut. Fungsi tampilan, proses bisnis, dan akses data tetap ada, tetapi belum dipisahkan menjadi layer yang jelas.

6 Tahap Cara Kerja Aplikasi Tanpa 3-Layer (Hal. 15 PDF)

Tahap 1 — Pengguna Mengisi Formulir

Pengguna memasukkan data nama, telepon, email, dan alamat pada form HTML.

Tahap 2 — Browser Mengirim Data ke index.php

Formulir dikirimkan ke file index.php yang sama menggunakan HTTP POST.

Tahap 3 — PHP Memproses Data

Kode PHP di dalam index.php melakukan validasi dan menjalankan query SQL.

Tahap 4 — MySQL Menyimpan Data

Data pelanggan yang valid dimasukkan ke dalam tabel pelanggan di MySQL.

Tahap 5 — PHP Mengambil Data Terbaru

File index.php menjalankan query SELECT untuk mengambil daftar terbaru.

Tahap 6 — Browser Menampilkan Data

Halaman dimuat ulang dan pengguna melihat tabel daftar pelanggan terbaru.

Konsep Pendekatan 2: 3-Layer Architecture

Pengertian Konsep & Penjelasan Detail Ketiga Lapisan (Sesuai Halaman 2–9 PDF)

Pengertian 3-Layer Web Application Architecture (Halaman 2 PDF):

- Konsep pengembangan aplikasi web dengan membagi sistem menjadi TIGA LAPISAN (LAYER) yang memiliki tugas & tanggung jawab berbeda.
- **Tiga lapisan tersebut adalah: (1) Presentation Layer, (2) Application / Business Logic Layer, dan (3) Data Layer.**

1. PRESENTATION LAYER

Lapisan Antarmuka (Frontend)

Pengertian:

Bagian aplikasi yang berinteraksi langsung dengan pengguna.

Tugas Utama:

- Menampilkan informasi (tabel/pesan)
- Menerima masukan form dari user
- Mengirim request ke backend via API

Teknologi yang Digunakan:

- HTML (struktur halaman)
- CSS (tampilan & desain)
- JavaScript (interaksi & Fetch API)
- (Opsional: React, Vue, Angular)

Intinya:

Bertanggung jawab terhadap apa yang dilihat & digunakan oleh pengguna.

2. BUSINESS LOGIC LAYER

Lapisan Logika Bisnis (Backend)

Pengertian:

Bagian yang menangani proses bisnis atau aturan utama aplikasi.

Tugas Utama:

- Menerima permintaan dari frontend
- Memvalidasi kelengkapan & aturan bisnis (misal: cek email unik)
- Mengambil keputusan & kirim JSON

Teknologi yang Digunakan:

- PHP Native (atau Laravel)
- Python, Java, Node.js, C#
- Komunikasi via REST API & JSON

Intinya:

Bertanggung jawab terhadap bagaimana aplikasi bekerja & mengambil keputusan.

3. DATA LAYER

Lapisan Pengelolaan Data (Database)

Pengertian:

Bagian yang berisi database dan mekanisme akses data.

Tugas Utama:

- Menyimpan data baru (INSERT)
- Mengambil data (SELECT / Cari)
- Memperbarui data (UPDATE)
- Menghapus data (DELETE)

Teknologi yang Digunakan:

- MySQL / MariaDB (PDO PHP)
- PostgreSQL, SQL Server, Oracle
- MongoDB

Intinya:

Bertanggung jawab terhadap penyimpanan dan pengelolaan data aplikasi.

Tanggung Jawab Layer & Perbandingan Karakteristik Dasar

Pembagian Tugas 3-Layer (Hal. 11 PDF) & Perbedaan Konsep Tanpa 3-Layer vs 3-Layer (Hal. 16 PDF)

Tabel Tanggung Jawab Ketiga Layer (Hal. 11 PDF)

Aspek	Presentation	Business Logic	Data Layer
Fungsi Utama	Tampilan (UI)	Proses bisnis	Penyimpanan data
Interaksi dgn	Pengguna	Frontend & DB	Backend
Contoh Tekno	HTML, CSS, JS	PHP, Python	MySQL
Folder Proyek	presentation/	business-logic/	data-access/
Tanggung Jawab	Menampilkan data	Mengolah data	Menyimpan data

5 Prinsip Dasar Arsitektur 3-Layer (Hal. 12 PDF):

1. Separation of concerns
2. Maintainability
3. Scalability
4. Security (Isolasi akses DB)
5. Reusability (REST API JSON)

Komunikasi: Presentation <-> REST API JSON <-> Logic <-> Data

Perbandingan Karakteristik Dasar (Hal. 16 PDF)

Aspek	Tanpa 3-Layer	Dengan 3-Layer
Struktur	Sederhana (tanpa batas layer)	Terpisah per layer
File program	Bisa 1 / beberapa file campur	Beberapa komponen/folder layer
Frontend	Menyatu dengan PHP	Terpisah (presentation/)
Business logic	Menyatu dgn kode aplikasi	Dipisahkan (business-logic/)
Database access	Langsung dari file PHP	Melalui komponen data access
Pemeliharaan	Lebih sulit jika kompleks	Lebih terstruktur
Pengembangan	Cepat untuk aplikasi kecil	Lebih terorganisasi
Pengujian	Sulit diuji per bagian/layer	Dapat dipisahkan per layer

Studi Kasus & Luaran 3: File Database SQL Lengkap

6 Fitur Wajib Aplikasi & Seluruh Isi File database/customer_management.sql (Baris 1–28)

6 Fitur Wajib yang Diimplementasikan

1. Tambah Pelanggan

Memasukkan data pelanggan baru (nama, email, telepon, alamat).

2. Tampil Pelanggan

Menampilkan seluruh daftar data pelanggan perusahaan.

3. Edit Pelanggan

Mengubah informasi pelanggan yang sudah terdaftar.

4. Hapus Pelanggan

Menghapus data pelanggan dari database MySQL.

5. Pencarian Pelanggan

Mencari pelanggan berdasarkan Nama atau Email.

6. Validasi Data

Memeriksa kelengkapan kolom dan keunikan alamat email.

LUARAN 3 — database/customer_management.sql (Baris 1–28 Lengkap)

```
001  -- ========================================================================
002  -- FILE DATABASE SQL: Customer Management System
003  -- Tugas Kelompok : Perbandingan Layered vs Non-Layered Web Architecture
004  -- Cara Import    : Buka phpMyAdmin -> Buat/Pilih DB -> Menu Import -> Pilih file ini
005  -- ========================================================================
006
007  CREATE DATABASE IF NOT EXISTS customer_management;
008  USE customer_management;
009
010  -- Membuat tabel pelanggan (sesuai studi kasus)
011  DROP TABLE IF EXISTS pelanggan;
012
013  CREATE TABLE pelanggan (
014      id INT AUTO_INCREMENT PRIMARY KEY,
015      nama VARCHAR(100) NOT NULL,
016      email VARCHAR(100) NOT NULL UNIQUE,
017      telepon VARCHAR(25) NOT NULL,
018      alamat TEXT NOT NULL,
019      created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
020  );
021
022  -- Memasukkan 5 data contoh pelanggan awal
023  INSERT INTO pelanggan (nama, email, telepon, alamat) VALUES
024  ('Andi Pratama', 'andi@nusantara.co.id', '0812-3456-7890', 'Jl. Setiabudhi No. 229, Kota B
       andung'),
025  ('Budi Santoso', 'budi@teknologimaju.id', '0813-9876-5432', 'Jl. Asia Afrika No. 88, Kota
       Bandung'),
026  ('Citra Lestari', 'citra@kreasidigital.com', '0821-5566-7788', 'Jl. Ir. H. Juanda (Dago) N
       o. 145, Kota Bandung'),
027  ('Dewi Anggraini', 'dewi@sinergiglobal.co.id', '0852-1122-3344', 'Jl. Soekarno Hatta No. 5
       12, Kota Bandung'),
028  ('Eko Prasetyo', 'eko@mitraniaga.id', '0878-4455-6677', 'Jl. Pasteur No. 32, Kota Bandung'
       );
```

Luaran 4 (A): Diagram Arsitektur Non-Layered

Ketentuan Halaman 20 PDF: Tanpa Pemisahan Layer Tegas (Diimplementasikan dalam 1 File index.php)

DIAGRAM ARSITEKTUR 1: NON-LAYERED (STRUKTUR FILE & HUBUNGAN KOMPONEN)

Tanpa Pemisahan Layer Tegas (Bisa 1 atau Beberapa File Tercampur; Studi Kasus Ini Menggunakan 1 File index.php)

STRUKTUR FOLDER & FILE (NON-LAYERED)

```
[Folder] non-layered-app/
 └── [File] index.php   (Implementasi 1 File)
      ├── [Baris 16-39]  Koneksi PDO MySQL
      ├── [Baris 44-160] Validasi PHP & Query SQL
      ├── [Baris 168-200] Kode Desain <style> CSS
      └── [Baris 202-304] Kode Tampilan HTML
```

Karakteristik Arsitektur Non-Layered:

1. Tanpa Pemisahan Layer Tegas

Bisa 1 atau beberapa file tanpa batas layer (di sini 1 file index.php).

2. Tampilan & Logika Menyatu

Formulir HTML dan tabel berada di file yang sama dengan kode PHP.

3. Validasi & Query Langsung

Pemeriksaan input dan query SQL langsung dijalankan saat submit.

4. Komunikasi Form HTTP Biasa

Menggunakan pengiriman form standar (POST/GET) tanpa REST API.

ISI DI DALAM FILE: non-layered-app/index.php (MENYATU)

BAGIAN TAMPILAN: HTML + CSS (Menyatu di Bagian Bawah index.php)

- Tag <style> CSS mengatur warna & tabel langsung di dalam index.php
- Tag <form> HTML & tabel <table> bercampur dengan <?php foreach (...) ?>

BAGIAN PROSES BISNIS: Validasi PHP (Menyatu di Bagian Tengah index.php)

- Blok if ($_SERVER['REQUEST_METHOD'] === 'POST') menerima input form
- Memeriksa kelengkapan kolom kosong & format email langsung di file yang sama

BAGIAN AKSES DATA: Koneksi PDO & Query SQL (Menyatu di index.php)

- `new PDO('mysql:host=localhost;dbname=customer_management', 'root', '')`
- Query SELECT, INSERT, UPDATE, DELETE ditulis langsung bersama kode form

Query SQL Langsung

Data Tabel pelanggan

DATABASE MYSQL: customer_management (File: database/customer_management.sql)

- Tabel `pelanggan`: id (PK), nama, email (UNIQUE), telepon, alamat, created_at
- **Diakses secara langsung dari file utama index.php menggunakan koneksi PDO MySQL**

Luaran 5 (A): Screenshot Code Aplikasi 1 (Non-Layered)

Ringkasan Visual Konsep Non-Layered pada non-layered-app/index.php (Menyatu dalam 1 File)

SCREENSHOT CODE 1: KONSEP NON-LAYERED ARCHITECTURE (1 FILE MONOLITIK)

File: non-layered-app/index.php — Koneksi Database, Validasi Proses Bisnis, Query SQL, CSS, dan HTML Menyatu dalam 1 File

non-layered-app/index.php (Seluruh Kode Menyatu dalam 1 File)

NON-LAYERED (MONOLITIK)

```
// ========================================================================
// BAGIAN 1: KONEKSI LANGSUNG KE DATABASE MYSQL (Di dalam index.php)
// ========================================================================
$koneksi = new PDO("mysql:host=localhost;dbname=customer_management", "root", "");

// ========================================================================
// BAGIAN 2: PROSES BISNIS (VALIDASI) & QUERY CRUD (Menyatu di index.php)
// ========================================================================
if (isset($_GET['hapus'])) {
    $stmt = $koneksi->prepare("DELETE FROM pelanggan WHERE id = ?");
    $stmt->execute([(int) $_GET['hapus']]);
}
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    // A. Validasi Kelengkapan Data
    if ($nama === '' || $email === '' || $telepon === '' || $alamat === '') {
        $pesan_error[] = "Semua kolom pelanggan wajib diisi.";
    }
    // B. Validasi Keunikan Email (Langsung query SQL di file tampilan)
    $stmtCek = $koneksi->prepare("SELECT id FROM pelanggan WHERE LOWER(email) = ?");
    $stmtCek->execute([$email]);
    if ($stmtCek->fetch()) {
        $pesan_error[] = "Email sudah terdaftar di database!";
    }
    // C. Eksekusi Query INSERT / UPDATE Langsung
    $stmt = $koneksi->prepare("INSERT INTO pelanggan (nama, email, telepon, alamat) VALUES (?,?,?,?)");
    $stmt->execute([$nama, $email, $telepon, $alamat]);
}
// Query Tampil & Pencarian Pelanggan
$daftar_pelanggan = $koneksi->query("SELECT * FROM pelanggan ORDER BY id DESC")->fetchAll();
?>
<!-- ========================================================================
     BAGIAN 3: TAMPILAN HTML & CSS (Menyatu langsung di bawah kode PHP)
     ======================================================================== -->
<!DOCTYPE html><html lang="id"><head><style> /* Kode CSS di sini */ </style></head>
```

Luaran 1 (Bagian 1/3): Source Code Lengkap Non-Layered (index.php)

File: non-layered-app/index.php (Baris 1 s/d 106 dari 310 Baris) — Koneksi PDO, Auto-Seed DB, & Validasi Input

Penjelasan: Koneksi database PDO MySQL (Baris 16-39) & proses bisnis validasi kelengkapan + cek email unik (Baris 44-106) ditulis langsung di file utama.

non-layered-app/index.php (Baris 1–53: Koneksi PDO & Auto-Seed SQL)

```php
<?php
/**
 * ============================================================================
 * APLIKASI 1: NON-LAYERED ARCHITECTURE (ARSITEKTUR MONOLITIK 1 FILE)
 * Studi Kasus : Customer Management System
 * Sesuai PDF  : Halaman 14, 15, 19, dan 20
 *
 * Ciri Utama  :
 * 1. Koneksi database MySQL dan query SQL ditulis langsung di dalam file ini.
 * 2. Proses bisnis (validasi kelengkapan & keunikan email) ada di file ini.
 * 3. Tampilan HTML digabungkan langsung dengan kode PHP di file ini.
 * 4. Tidak menggunakan REST API (menggunakan submit form biasa).
 * ============================================================================
 */

// ============================================================================
// BAGIAN 1: KONEKSI LANGSUNG KE DATABASE MYSQL
// ============================================================================
$host     = "localhost";
$dbname   = "customer_management";
$username = "root";
$password = "";

$options = [
    PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
];

try {
    $koneksi = new PDO("mysql:host=$host;dbname=$dbname;charset=utf8mb4", $username, $pass
  word, $options);
} catch (PDOException $e) {
    // Jika database customer_management belum di-import di MySQL, otomatis jalankan file
  database/customer_management.sql
    $sqlFile = dirname(__DIR__) . "/database/customer_management.sql";
    if (strpos($e->getMessage(), "1049") !== false && file_exists($sqlFile)) {
        $pdoSetup = new PDO("mysql:host=$host;charset=utf8mb4", $username, $password, $opt
  ions);
        $pdoSetup->exec(file_get_contents($sqlFile));
        $koneksi = new PDO("mysql:host=$host;dbname=$dbname;charset=utf8mb4", $username, $
  password, $options);
    } else {
        die("Koneksi MySQL gagal. Pastikan MySQL di XAMPP/Laragon sudah dinyalakan. Detail
  : " . $e->getMessage());
    }
}

// ============================================================================
// BAGIAN 2: PROSES BISNIS (VALIDASI) & QUERY CRUD KE DATABASE MYSQL
// ============================================================================
$pesan_error  = [];
$pesan_sukses = "";

// Variabel penampung nilai form (untuk Tambah maupun Edit)
$id_edit = "";
$nama    = "";
$email   = "";
$telepon = "";
```

non-layered-app/index.php (Baris 54–106: Hapus Data & Validasi Input POST)

```php
$alamat  = "";

// --- FITUR 4: HAPUS PELANGGAN (Saat tombol Hapus diklik) ---
if (isset($_GET['hapus'])) {
    $id_hapus = (int) $_GET['hapus'];
    $stmt = $koneksi->prepare("DELETE FROM pelanggan WHERE id = ?");
    $stmt->execute([$id_hapus]);
    $pesan_sukses = "Data pelanggan berhasil dihapus dari database.";
}

// --- PERSIAPAN FITUR 3: AMBIL DATA DARI DATABASE UNTUK FORM EDIT ---
if (isset($_GET['edit'])) {
    $id_ambil = (int) $_GET['edit'];
    $stmt = $koneksi->prepare("SELECT * FROM pelanggan WHERE id = ?");
    $stmt->execute([$id_ambil]);
    $data_lama = $stmt->fetch();

    if ($data_lama) {
        $id_edit = $data_lama['id'];
        $nama    = $data_lama['nama'];
        $email   = $data_lama['email'];
        $telepon = $data_lama['telepon'];
        $alamat  = $data_lama['alamat'];
    }
}

// --- FITUR 1, 3, & 6: SIMPAN / UPDATE PELANGGAN & VALIDASI DATA ---
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $id_edit = trim($_POST['id_edit'] ?? '');
    $nama    = trim($_POST['nama'] ?? '');
    $email   = strtolower(trim($_POST['email'] ?? ''));
    $telepon = trim($_POST['telepon'] ?? '');
    $alamat  = trim($_POST['alamat'] ?? '');

    // A. Validasi Kelengkapan Data (Semua kolom wajib diisi)
    if ($nama === '') {
        $pesan_error[] = "Nama pelanggan wajib diisi.";
    }
    if ($email === '') {
        $pesan_error[] = "Email pelanggan wajib diisi.";
    } elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        $pesan_error[] = "Format alamat email tidak valid.";
    }
    if ($telepon === '') {
        $pesan_error[] = "Nomor telepon wajib diisi.";
    }
    if ($alamat === '') {
        $pesan_error[] = "Alamat pelanggan wajib diisi.";
    }

    // B. Validasi Keunikan Email (Cek langsung ke tabel pelanggan di MySQL)
    if (empty($pesan_error)) {
        if ($id_edit !== '') {
```

Luaran 1 (Bagian 2/3): Source Code Lengkap Non-Layered (index.php)

File: non-layered-app/index.php (Baris 107 s/d 208 dari 310 Baris) — Query SQL CRUD, Pencarian, & Desain CSS

Penjelasan: Query SQL INSERT, UPDATE, SELECT pencarian (Baris 107-160) langsung menyatu dengan kode desain <style> CSS (Baris 168-200) di file yang sama.

non-layered-app/index.php (Baris 107–158: Query SQL INSERT, UPDATE, & SELECT)

```
            // Saat Edit: cek apakah email dipakai oleh pelanggan selain dirinya sendiri
            $stmtCek = $koneksi->prepare("SELECT id FROM pelanggan WHERE LOWER(email) = ? AND id != ?");
            $stmtCek->execute([$email, (int) $id_edit]);
        } else {
            // Saat Tambah Baru: cek apakah email sudah ada di tabel pelanggan
            $stmtCek = $koneksi->prepare("SELECT id FROM pelanggan WHERE LOWER(email) = ?");
            $stmtCek->execute([$email]);
        }

        if ($stmtCek->fetch()) {
            $pesan_error[] = "Email '$email' sudah terdaftar di database. Gunakan email yang unik!";
        }
    }

    // C. Jika Validasi Lolos -> Jalankan Query INSERT atau UPDATE ke MySQL
    if (empty($pesan_error)) {
        if ($id_edit !== '') {
            // FITUR 3: EDIT PELANGGAN (UPDATE)
            $stmt = $koneksi->prepare("UPDATE pelanggan SET nama = ?, email = ?, telepon = ?, alamat = ? WHERE id = ?");
            $stmt->execute([$nama, $email, $telepon, $alamat, (int) $id_edit]);
            $pesan_sukses = "Data pelanggan '$nama' berhasil diperbarui!";
        } else {
            // FITUR 1: TAMBAH PELANGGAN BARU (INSERT)
            $stmt = $koneksi->prepare("INSERT INTO pelanggan (nama, email, telepon, alamat) VALUES (?, ?, ?, ?)");
            $stmt->execute([$nama, $email, $telepon, $alamat]);
            $pesan_sukses = "Pelanggan baru '$nama' berhasil ditambahkan ke database!";
        }

        // Kosongkan form kembali setelah berhasil disimpan
        $id_edit = $nama = $email = $telepon = $alamat = "";
    }
}

// --- FITUR 2 & 5: TAMPIL PELANGGAN & PENCARIAN DARI DATABASE MYSQL ---
$kata_kunci = trim($_GET['cari'] ?? '');
if ($kata_kunci !== '') {
    $stmt = $koneksi->prepare("SELECT * FROM pelanggan WHERE nama LIKE ? OR email LIKE ? ORDER BY id DESC");
    $stmt->execute(["%$kata_kunci%", "%$kata_kunci%"]);
    $daftar_pelanggan = $stmt->fetchAll();
} else {
    $stmt = $koneksi->query("SELECT * FROM pelanggan ORDER BY id DESC");
    $daftar_pelanggan = $stmt->fetchAll();
}
?>
<!-- ========================================================================
     BAGIAN 3: TAMPILAN HTML & CSS (Menyatu langsung dengan kode PHP di atas)
     ======================================================================== -->
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <title>Customer Management System - Non-Layered Architecture</title>
```

non-layered-app/index.php (Baris 159–208: Tag <head>, <style> CSS, & Header HTML)

```
    <style>
        * { box-sizing: border-box; font-family: 'Segoe UI', Arial, sans-serif; }
        body { background: #f4f6f9; margin: 0; padding: 0; color: #1e293b; }
        .header { background: #b45309; color: white; padding: 18px 30px; }
        .header h1 { margin: 0; font-size: 22px; }
        .header p { margin: 4px 0 0; font-size: 14px; opacity: 0.9; }
        .container { max-width: 1150px; margin: 20px auto; padding: 0 20px; }
        .info-box { background: #fffbeb; border-left: 5px solid #d97706; padding: 12px 16px; margin-bottom: 18px; border-radius: 6px; font-size: 14px; }
        .alert-error { background: #fef2f2; border-left: 5px solid #dc2626; color: #991b1b; padding: 12px 16px; margin-bottom: 18px; border-radius: 6px; }
        .alert-success { background: #f0fdf4; border-left: 5px solid #16a34a; color: #166534; padding: 12px 16px; margin-bottom: 18px; border-radius: 6px; font-weight: bold; }
        .grid { display: grid; grid-template-columns: 350px 1fr; gap: 20px; align-items: start; }
        @media (max-width: 850px) { .grid { grid-template-columns: 1fr; } }
        .card { background: white; border: 1px solid #cbd5e1; border-radius: 8px; padding: 18px; box-shadow: 0 1px 3px rgba(0,0,0,0.05); }
        .card h2 { margin-top: 0; font-size: 18px; border-bottom: 1px solid #e2e8f0; padding-bottom: 10px; }
        .form-group { margin-bottom: 12px; }
        .form-group label { display: block; font-weight: 600; font-size: 13px; margin-bottom: 5px; }
        .form-group input, .form-group textarea { width: 100%; padding: 8px 10px; border: 1px solid #cbd5e1; border-radius: 5px; font-size: 14px; }
        .btn { display: inline-block; padding: 8px 14px; border: none; border-radius: 5px; cursor: pointer; font-weight: 600; font-size: 13px; text-decoration: none; }
        .btn-utama { background: #d97706; color: white; }
        .btn-utama:hover { background: #b45309; }
        .btn-batal { background: #e2e8f0; color: #334155; }
        .btn-edit { background: #dbeafe; color: #1d4ed8; padding: 5px 10px; }
        .btn-hapus { background: #fee2e2; color: #dc2626; padding: 5px 10px; }
        .search-form { display: flex; gap: 8px; margin-bottom: 14px; }
        .search-form input { flex: 1; padding: 8px 10px; border: 1px solid #cbd5e1; border-radius: 5px; }
        table { width: 100%; border-collapse: collapse; font-size: 14px; }
        th, td { border-bottom: 1px solid #e2e8f0; padding: 10px; text-align: left; vertical-align: top; }
        th { background: #f8fafc; font-size: 12px; text-transform: uppercase; color: #475569; }
    </style>
</head>
<body>

<div class="header">
    <h1>Customer Management System — Aplikasi 1: Non-Layered Architecture</h1>
    <p>Pendekatan Monolitik Sederhana: HTML, Validasi Proses Bisnis, dan Query Database Menyatu dalam 1 File (index.php)</p>
</div>

<div class="container">
    <div class="info-box">
        <strong>Info Arsitektur:</strong> Pada aplikasi ini, seluruh proses mulai dari menampilkan form, menerima input, memvalidasi data, mengakses database MySQL, hingga menampilkan tabel pelanggan berada di dalam satu file <code>index.php</code>.
    </div>

    <!-- Menampilkan Pesan Error Validasi (Jika Ada) -->
    <?php if (!empty($pesan_error)): ?>
        <div class="alert-error">
            <strong>Validasi Gagal:</strong>
            <ul style="margin: 6px 0 0; padding-left: 20px;">
                <?php foreach ($pesan_error as $err): ?>
                    <li><?= htmlspecialchars($err) ?></li>
                <?php endforeach; ?>
```

Luaran 1 (Bagian 3/3): Source Code Lengkap Non-Layered (index.php)

File: non-layered-app/index.php (Baris 209 s/d 310 Selesai) — Formulir HTML, Alert Validasi, & Tabel Pelanggan

Penjelasan: Kode tampilan <form> HTML dan perulangan PHP <?php foreach ($daftarPelanggan as $p): ?> (Baris 209-310) berada di bagian bawah file index.php.

non-layered-app/index.php (Baris 209–260: Alert Validasi & Form Tambah/Edit HTML)

```
209             </ul>
210         </div>
211     <?php endif; ?>
212
213     <!-- Menampilkan Pesan Sukses (Jika Ada) -->
214     <?php if ($pesan_sukses !== ""): ?>
215         <div class="alert-success">
216             &#10004; <?= htmlspecialchars($pesan_sukses) ?>
217         </div>
218     <?php endif; ?>
219
220     <div class="grid">
221         <!-- KOLOM KIRI: FORM TAMBAH / EDIT PELANGGAN -->
222         <div class="card">
223             <h2><?= $id_edit !== '' ? "Edit Pelanggan #$id_edit" : "Tambah Pelanggan Baru"
     ?></h2>
224             <form method="POST" action="index.php">
225                 <input type="hidden" name="id_edit" value="<?= htmlspecialchars((string)$i
    d_edit) ?>">
226
227                 <div class="form-group">
228                     <label>Nama Pelanggan *</label>
229                     <input type="text" name="nama" value="<?= htmlspecialchars($nama) ?>"
    placeholder="Contoh: Andi Pratama">
230                 </div>
231
232                 <div class="form-group">
233                     <label>Email Pelanggan (Unik) *</label>
234                     <input type="text" name="email" value="<?= htmlspecialchars($email) ?>
    " placeholder="Contoh: andi@email.com">
235                 </div>
236
237                 <div class="form-group">
238                     <label>Nomor Telepon *</label>
239                     <input type="text" name="telepon" value="<?= htmlspecialchars($telepon
    ) ?>" placeholder="Contoh: 0812-3456-7890">
240                 </div>
241
242                 <div class="form-group">
243                     <label>Alamat Lengkap *</label>
244                     <textarea name="alamat" rows="3" placeholder="Masukkan alamat lengkap.
    .."><?= htmlspecialchars($alamat) ?></textarea>
245                 </div>
246
247                 <button type="submit" class="btn btn-utama">
248                     <?= $id_edit !== '' ? "Simpan Perubahan" : "+ Tambah Pelanggan" ?>
249                 </button>
250                 <?php if ($id_edit !== ''): ?>
251                     <a href="index.php" class="btn btn-batal">Batal</a>
252                 <?php endif; ?>
253             </form>
254         </div>
255
256         <!-- KOLOM KANAN: PENCARIAN & TABEL DATA PELANGGAN -->
257         <div class="card">
258             <h2>Daftar Pelanggan (Total: <?= count($daftar_pelanggan) ?>)</h2>
259
260             <!-- Form Pencarian Berdasarkan Nama atau Email -->
```

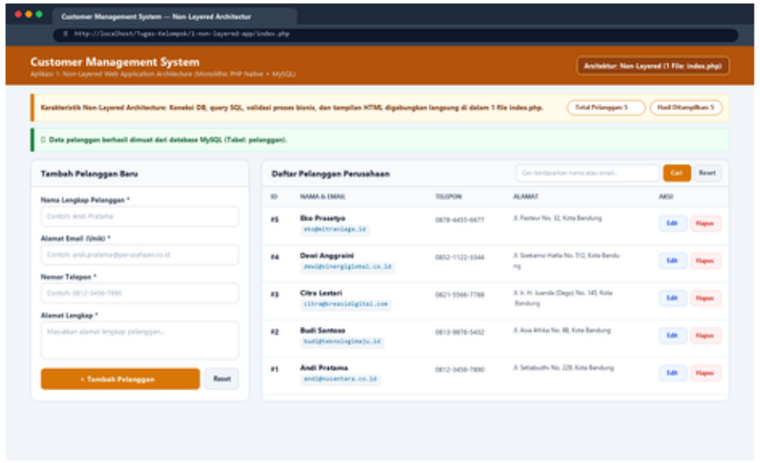
non-layered-app/index.php (Baris 261–310 Selesai: Form Pencarian & Tabel HTML)

```
261             <form method="GET" action="index.php" class="search-form">
262                 <input type="text" name="cari" value="<?= htmlspecialchars($kata_kunci) ?>
    " placeholder="Cari berdasarkan nama atau email...">
263                 <button type="submit" class="btn btn-utama">Cari</button>
264                 <?php if ($kata_kunci !== ''): ?>
265                     <a href="index.php" class="btn btn-batal">Reset</a>
266                 <?php endif; ?>
267             </form>
268
269             <table>
270                 <thead>
271                     <tr>
272                         <th>ID</th>
273                         <th>Nama &amp;amp; Email</th>
274                         <th>Telepon</th>
275                         <th>Alamat</th>
276                         <th style="width: 120px;">Aksi</th>
277                     </tr>
278                 </thead>
279                 <tbody>
280                     <?php if (empty($daftar_pelanggan)): ?>
281                         <tr>
282                             <td colspan="5" style="text-align: center; padding: 20px; colo
    r: #64748b;">
283                                 Data pelanggan tidak ditemukan.
284                             </td>
285                         </tr>
286                     <?php else: ?>
287                         <?php foreach ($daftar_pelanggan as $row): ?>
288                             <tr>
289                                 <td><strong>#<?= $row['id'] ?></strong></td>
290                                 <td>
291                                     <strong><?= htmlspecialchars($row['nama']) ?></strong>
    <br>
292                                     <small style="color: #0369a1;"><?= htmlspecialchars($r
    ow['email']) ?></small>
293                                 </td>
294                                 <td><?= htmlspecialchars($row['telepon']) ?></td>
295                                 <td><?= htmlspecialchars($row['alamat']) ?></td>
296                                 <td>
297                                     <a href="index.php?edit=<?= $row['id'] ?>" class="btn
    btn-edit">Edit</a>
298                                     <a href="index.php?hapus=<?= $row['id'] ?>" class="btn
     btn-hapus" onclick="return confirm('Hapus pelanggan ini?')">Hapus</a>
299                                 </td>
300                             </tr>
301                         <?php endforeach; ?>
302                     <?php endif; ?>
303                 </tbody>
304             </table>
305         </div>
306     </div>
307 </div>
308
309 </body>
310 </html>
```

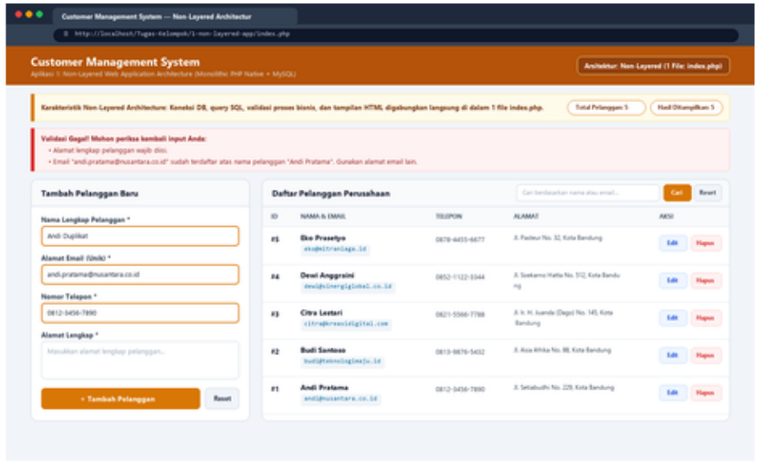
Luaran 5 (B): Screenshot Tampilan Web Aplikasi 1 (Non-Layered)

Lengkap 3 Skenario Demo pada non-layered-app/index.php: Dashboard CRUD, Validasi Email Unik, & Pencarian

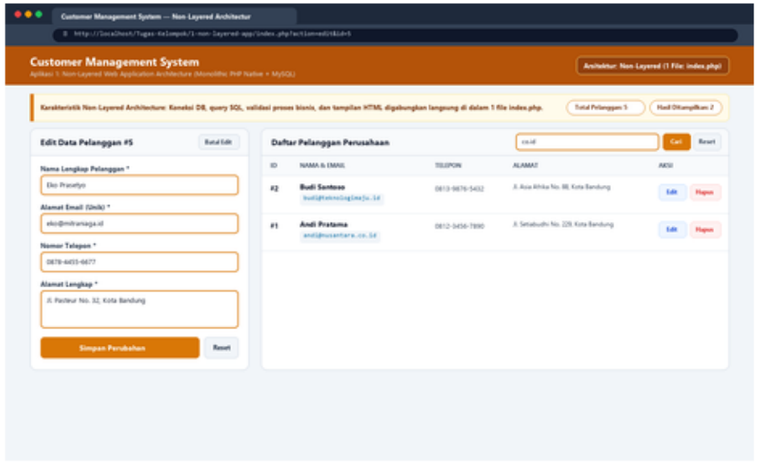
1. Dashboard Utama & Tabel CRUD Pelanggan



2. Validasi Kolom Kosong & Penolakan Email Duplikat



3. Fitur Edit & Pencarian Berdasarkan Nama / Email



Luaran 4 (B): Diagram Arsitektur 3-Layer (Pemisahan Folder)

Ketentuan Halaman 21 PDF: Pemisahan Tegas presentation/ <-> business-logic/ <-> data-access/ via REST API JSON

DIAGRAM ARSITEKTUR 2: 3-LAYER ARCHITECTURE (PEMISAHAN FOLDER & FILE)

Pemisahan Tegas ke 3 Folder: presentation/ (HTML/CSS/JS) <-> business-logic/ (PHP API/Service) <-> data-access/ (PDO SQL)

LAYER 1: PRESENTATION LAYER

Folder: 3-layered-app/presentation/

File: index.html (Struktur Halaman)
- Menampilkan form input & tabel pelanggan
- **Murni HTML (0% kode PHP / 0% query SQL)**

File: assets/css/style.css (Desain UI)
- File CSS terpisah untuk mengatur tampilan
- Di-link oleh index.html (<link rel='stylesheet'>)

File: assets/js/app.js (Interaksi & Fetch)
- Mengirim & mengambil data ke Backend
- Menggunakan fetch() REST API & JSON
- Memperbarui isi tabel tanpa reload halaman

Tugas Folder presentation/:
Hanya mengurus tampilan & interaksi user.

HTTP Request

Format JSON

JSON Response

LAYER 2: BUSINESS LOGIC LAYER

Folder: 3-layered-app/business-logic/

File: api/customers.php (REST API Endpoint)
- Pintu komunikasi antara JS dan PHP
- Menerima input JSON (json_decode)
- Memanggil CustomerService.php
- Mengirim hasil JSON (json_encode)

File: CustomerService.php (Aturan Bisnis)
- **Fungsi validasiData():**
  1. Cek kelengkapan nama, email, telp, alamat
  2. Cek format email (FILTER_VALIDATE_EMAIL)
  3. Cek keunikan email ke CustomerRepository
- **Bebas dari kode HTML & bebas dari SQL!**

Tugas Folder business-logic/:
Mengatur logika bisnis, validasi, & REST API.

Panggil Fungsi

Return Array

LAYER 3: DATA LAYER

Folder: 3-layered-app/data-access/

File: CustomerRepository.php (SQL)
- `findAll()` -> `SELECT * FROM pelanggan`
- `search()` -> `SELECT ... WHERE LIKE`
- `findByEmail()` -> Cek email unik di DB
- `create()` -> `INSERT INTO pelanggan`
- `update()` -> `UPDATE pelanggan SET`
- `delete()` -> `DELETE FROM pelanggan`

File: Database.php (Koneksi MySQL)
- Membuka koneksi PDO ke MySQL
- Database: customer_management
- Digunakan oleh CustomerRepository

Tugas Folder data-access/:
Satu-satunya folder yang akses MySQL.

DATABASE MYSQL: customer_management

`File: database/customer_management.sql`

- Tabel `pelanggan` (5 data awal)
- **Aman & terisolasi di belakang Data Layer**

RINGKASAN PEMISAHAN FILE/FOLDER PADA 3-LAYER ARCHITECTURE:

```
[Folder 1] 3-layered-app/presentation/     --> index.html + style.css + app.js (Frontend/UI)
[Folder 2] 3-layered-app/business-logic/   --> CustomerService.php + api/customers.php (Backend/Logic)
[Folder 3] 3-layered-app/data-access/      --> Database.php + CustomerRepository.php (Database/SQL)
```

Luaran 5 (C): Screenshot Code Aplikasi 2 (3-Layer Architecture)

Ringkasan Visual Pemisahan Kode ke 3 Folder: app.js (Presentation), CustomerService.php (Logic), & CustomerRepository.php (Data)

SCREENSHOT CODE 2: KONSEP 3-LAYER ARCHITECTURE (TERPISAH 3 FOLDER LAYER)

Pemisahan Tegas: Presentation Layer (app.js) <-> Business Logic Layer (CustomerService.php & api) <-> Data Layer (CustomerRepository.php)

presentation/app.js — LAYER 1: PRESENTATION

```
1   // Folder: 3-layered-app/presentation/
2   // Murni Frontend (Tanpa PHP & Tanpa SQL)
8   const URL_API = '../business-logic/api/customers.php';
9
11  // 1. Ambil Data via GET REST API (JSON)
12  async function muatDataPelanggan(kataKunci = '') {
18    const response = await fetch(url);
19    const hasil = await response.json();
30    tbody.innerHTML = hasil.data.map(...);
45  }
46
48  // 2. Kirim Data via POST/PUT JSON
60  const response = await fetch(url, {
61    method: idEdit ? 'PUT' : 'POST',
62    headers: {'Content-Type':'application/json'},
63    body: JSON.stringify(dataKirim)
64  });
66  const hasil = await response.json();
```

CustomerService.php — LAYER 2: BUSINESS LOGIC

```
1   // Folder: 3-layered-app/business-logic/
2   // Aturan Bisnis, Validasi, & REST API JSON
14  class CustomerService {
56    public function createCustomer($input) {
58      $data = $this->rapikanInput($input);
59      $errors = $this->validasiData($data);
62      if (!empty($errors)) {
63        return ['status_code' => 422,
64                'success' => false,
65                'errors' => $errors];
67      }
70      $baru = $this->repo->create($data);
72      return ['status_code' => 201,
73              'success' => true,
74              'data' => $baru];
76    }
150   // Validasi kelengkapan & email unik
151   private function validasiData($data) {...}
180 }
```

CustomerRepository.php — LAYER 3: DATA LAYER

```
1   // Folder: 3-layered-app/data-access/
2   // Khusus Koneksi PDO & Query SQL MySQL
12  class CustomerRepository {
22    public function findAll() {
24      return $this->db->query(
25        'SELECT * FROM pelanggan ORDER BY id DESC'
26      )->fetchAll();
27    }
46    public function findByEmail($email) {
52      $stmt = $this->db->prepare(
53        'SELECT * FROM pelanggan WHERE email = ?'
54      );
57    }
60    public function create($data) {
62      $stmt = $this->db->prepare(
63        'INSERT INTO pelanggan VALUES (?,?,?,?)'
64      );
65      $stmt->execute([...]);
70    }
95  }
```

Luaran 2 (File 1/7 — Presentation Layer): presentation/index.html

File: 3-layered-app/presentation/index.html (Baris 1 s/d 108 Lengkap) — Murni Struktur Tampilan HTML (0% Kode PHP/SQL)

Presentation Layer (File 1): index.html hanya menangani tampilan form input, tabel pelanggan, & panel monitor JSON tanpa kode PHP maupun SQL.

presentation/index.html (Baris 1–54: Header, Banner Layer, & Form Input)

```
<!-- ========================================================================
     LAYER 1: PRESENTATION LAYER — presentation/index.html
     Tugas : Menampilkan antarmuka pengguna (Form & Tabel Pelanggan).
             File ini murni HTML (tanpa kode PHP maupun query SQL).
     ======================================================================== -->
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <title>Customer Management System - 3-Layer Architecture</title>
    <link rel="stylesheet" href="assets/css/style.css">
</head>
<body>

<div class="header">
    <h1>Customer Management System — Aplikasi 2: 3-Layer Architecture</h1>
    <p>Pemisahan Tegas: Presentation Layer (HTML/JS) &harr; Business Logic Layer (REST API JSON) &harr; Data Layer (MySQL)</p>
</div>

<div class="container">
    <div class="info-box">
        <strong>Info Arsitektur:</strong> Halaman ini adalah <strong>Presentation Layer</strong> murni. Setiap aksi (Tambah, Tampil, Edit, Hapus, Cari, &amp;amp; Validasi) dikirim ke <strong>Business Logic Layer</strong> menggunakan <strong>REST API &amp;amp; JSON</strong>.
    </div>

    <!-- Kotak Pesan Error Validasi (dari JSON Business Logic Layer) -->
    <div id="box-error" class="alert alert-error">
        <strong>Validasi Gagal (Respons JSON dari Business Logic Layer):</strong>
        <ul id="list-error" style="margin: 6px 0 0; padding-left: 20px;"></ul>
    </div>

    <!-- Kotak Pesan Sukses -->
    <div id="box-sukses" class="alert alert-success"></div>

    <div class="grid">
        <!-- KOLOM KIRI: FORM TAMBAH / EDIT PELANGGAN -->
        <div class="card">
            <h2 id="judul-form">Tambah Pelanggan Baru</h2>
            <form id="form-pelanggan">
                <input type="hidden" id="id_edit" value="">

                <div class="form-group">
                    <label>Nama Pelanggan *</label>
                    <input type="text" id="nama" placeholder="Contoh: Andi Pratama">
                </div>

                <div class="form-group">
                    <label>Email Pelanggan (Unik) *</label>
                    <input type="text" id="email" placeholder="Contoh: andi@email.com">
                </div>

                <div class="form-group">
                    <label>Nomor Telepon *</label>
                    <input type="text" id="telepon" placeholder="Contoh: 0812-3456-7890">
                </div>
```

presentation/index.html (Baris 55–108 Selesai: Kotak Pencarian, Tabel, & Panel JSON)

```

                <div class="form-group">
                    <label>Alamat Lengkap *</label>
                    <textarea id="alamat" rows="3" placeholder="Masukkan alamat lengkap..."></textarea>
                </div>

                <button type="submit" id="tombol-simpan" class="btn btn-utama">+ Tambah Pelanggan</button>
                <button type="button" id="tombol-batal" class="btn btn-batal" style="display: none;" onclick="resetForm()">Batal</button>
            </form>
        </div>

        <!-- KOLOM KANAN: PENCARIAN & TABEL DATA PELANGGAN -->
        <div class="card">
            <h2>Daftar Pelanggan (Total: <span id="jumlah-data">0</span>)</h2>

            <!-- Form Pencarian Berdasarkan Nama atau Email -->
            <form id="form-cari" class="search-form">
                <input type="text" id="input-cari" placeholder="Cari berdasarkan nama atau email...">
                <button type="submit" class="btn btn-utama">Cari</button>
                <button type="button" class="btn btn-batal" onclick="resetPencarian()">Reset</button>
            </form>

            <table>
                <thead>
                    <tr>
                        <th>ID</th>
                        <th>Nama &amp; Email</th>
                        <th>Telepon</th>
                        <th>Alamat</th>
                        <th style="width: 120px;">Aksi</th>
                    </tr>
                </thead>
                <tbody id="isi-tabel">
                    <tr>
                        <td colspan="5" style="text-align: center; padding: 20px;">Memuat data dari REST API...</td>
                    </tr>
                </tbody>
            </table>
        </div>
    </div>

    <!-- PANEL BUKTI KOMUNIKASI REST API & JSON ANTAR LAYER -->
    <div class="json-box">
        <div class="json-box-title">
            <span>Monitor Komunikasi REST API &amp;amp; JSON (Presentation Layer &harr; Business Logic Layer)</span>
            <span id="info-endpoint">GET ../business-logic/api/customers.php</span>
        </div>
        <pre id="isi-json">Menunggu respons JSON...</pre>
    </div>
</div>

<script src="assets/js/app.js"></script>
</body>
</html>
```

Luaran 2 (File 2/7 — Presentation Layer Bagian 1/2): style.css

File: 3-layered-app/presentation/assets/css/style.css (Baris 1 s/d 108 dari 214 Baris) — Desain Visual & Layout

Presentation Layer (File 2): style.css dipisahkan di dalam folder presentation/assets/css/ sehingga desainer aman mengubah UI tanpa menyentuh kode backend.

presentation/assets/css/style.css (Baris 1–54: Reset, Container, & Header Badge)

```
/* ============================================================================
 * LAYER 1: PRESENTATION LAYER — assets/css/style.css
 * Tugas : Mengatur tampilan visual halaman web pada aplikasi 3-Layer.
 * ============================================================================ */

* {
    box-sizing: border-box;
    font-family: 'Segoe UI', Arial, sans-serif;
}

body {
    background: #f4f6f9;
    margin: 0;
    padding: 0;
    color: #1e293b;
}

.header {
    background: #1e3a8a;
    color: white;
    padding: 18px 30px;
}

.header h1 {
    margin: 0;
    font-size: 22px;
}

.header p {
    margin: 4px 0 0;
    font-size: 14px;
    opacity: 0.9;
}

.container {
    max-width: 1150px;
    margin: 20px auto;
    padding: 0 20px 40px;
}

.info-box {
    background: #eff6ff;
    border-left: 5px solid #2563eb;
    padding: 12px 16px;
    margin-bottom: 18px;
    border-radius: 6px;
    font-size: 14px;
}

.alert {
    padding: 12px 16px;
    margin-bottom: 18px;
    border-radius: 6px;
    display: none;
```

presentation/assets/css/style.css (Baris 55–108: Grid Layout, Card, & Form Input)

```
}

.alert-error {
    background: #fef2f2;
    border-left: 5px solid #dc2626;
    color: #991b1b;
}

.alert-success {
    background: #f0fdf4;
    border-left: 5px solid #16a34a;
    color: #166534;
    font-weight: bold;
}

.grid {
    display: grid;
    grid-template-columns: 350px 1fr;
    gap: 20px;
    align-items: start;
}

@media (max-width: 850px) {
    .grid {
        grid-template-columns: 1fr;
    }
}

.card {
    background: white;
    border: 1px solid #cbd5e1;
    border-radius: 8px;
    padding: 18px;
    box-shadow: 0 1px 3px rgba(0, 0, 0, 0.05);
}

.card h2 {
    margin-top: 0;
    font-size: 18px;
    border-bottom: 1px solid #e2e8f0;
    padding-bottom: 10px;
}

.form-group {
    margin-bottom: 12px;
}

.form-group label {
    display: block;
    font-weight: 600;
    font-size: 13px;
    margin-bottom: 5px;
}

```

Luaran 2 (File 2/7 — Presentation Layer Bagian 2/2): style.css

File: 3-layered-app/presentation/assets/css/style.css (Baris 109 s/d 214 Selesai) — Tabel, Tombol, & Panel JSON

Presentation Layer (File 2 Lanjutan): Mengatur tampilan tombol aksi CRUD, alert notifikasi, tabel responsif, serta panel monitor respons REST API JSON.

presentation/assets/css/style.css (Baris 109–162: Tombol Aksi, Alert, & Search Bar)

```
.form-group input,
.form-group textarea {
    width: 100%;
    padding: 8px 10px;
    border: 1px solid #cbd5e1;
    border-radius: 5px;
    font-size: 14px;
}

.btn {
    display: inline-block;
    padding: 8px 14px;
    border: none;
    border-radius: 5px;
    cursor: pointer;
    font-weight: 600;
    font-size: 13px;
    text-decoration: none;
}

.btn-utama {
    background: #2563eb;
    color: white;
}

.btn-utama:hover {
    background: #1d4ed8;
}

.btn-batal {
    background: #e2e8f0;
    color: #334155;
}

.btn-edit {
    background: #dbeafe;
    color: #1d4ed8;
    padding: 5px 10px;
}

.btn-hapus {
    background: #fee2e2;
    color: #dc2626;
    padding: 5px 10px;
}

.search-form {
    display: flex;
    gap: 8px;
    margin-bottom: 14px;
}

.search-form input {
    flex: 1;
```

presentation/assets/css/style.css (Baris 163–214 Selesai: Tabel Data & Monitor JSON)

```
    padding: 8px 10px;
    border: 1px solid #cbd5e1;
    border-radius: 5px;
}

table {
    width: 100%;
    border-collapse: collapse;
    font-size: 14px;
}

th, td {
    border-bottom: 1px solid #e2e8f0;
    padding: 10px;
    text-align: left;
    vertical-align: top;
}

th {
    background: #f8fafc;
    font-size: 12px;
    text-transform: uppercase;
    color: #475569;
}

.json-box {
    margin-top: 20px;
    background: #0f172a;
    color: #a7f3d0;
    padding: 14px 18px;
    border-radius: 8px;
    font-family: Consolas, monospace;
    font-size: 12px;
}

.json-box-title {
    color: #93c5fd;
    font-weight: bold;
    margin-bottom: 8px;
    display: flex;
    justify-content: space-between;
}

.json-box pre {
    margin: 0;
    background: #1e293b;
    padding: 10px;
    border-radius: 5px;
    max-height: 150px;
    overflow: auto;
    white-space: pre-wrap;
}
```

Luaran 2 (File 3/7 — Presentation Layer Bagian 1/2): app.js

File: 3-layered-app/presentation/assets/js/app.js (Baris 1 s/d 90 dari 176 Baris) — Fetch GET REST API & Render Tabel

Presentation Layer (File 3): app.js bertugas menghubungkan antarmuka pengguna dengan Business Logic Layer melalui HTTP Request Fetch API & JSON.

presentation/assets/js/app.js (Baris 1–45: Konfigurasi Endpoint API & Monitor JSON)

```
/**
 * ============================================================================
 * LAYER 1: PRESENTATION LAYER — assets/js/app.js
 * Tugas : Menghubungkan tampilan HTML dengan Backend (Business Logic Layer)
 *         menggunakan REST API (fetch) dan format data JSON.
 * ============================================================================
 */

const URL_API = '../business-logic/api/customers.php';

// 1. Fungsi untuk mengambil & menampilkan daftar pelanggan (GET)
async function muatDataPelanggan(kataKunci = '') {
    let url = URL_API;
    if (kataKunci !== '') {
        url += '?q=' + encodeURIComponent(kataKunci);
    }

    const response = await fetch(url);
    const hasil = await response.json();

    // Tampilkan log JSON di panel bawah untuk bukti komunikasi REST API
    tampilkanLogJson('GET ' + url + ' (HTTP ' + response.status + ')', hasil);

    if (!hasil.success) {
        tampilkanError(hasil.errors || [hasil.message]);
    }

    const tbody = document.getElementById('isi-tabel');
    document.getElementById('jumlah-data').textContent = hasil.data ? hasil.data.length :
  0;

    if (!hasil.data || hasil.data.length === 0) {
        tbody.innerHTML = '<tr><td colspan="5" style="text-align:center; padding:20px;">Da
    ta pelanggan tidak ditemukan.</td></tr>';
        return;
    }

    tbody.innerHTML = hasil.data.map(row => `
        <tr>
            <td><strong>#${row.id}</strong></td>
            <td>
                <strong>${ row.nama }</strong><br>
                <small style="color:#1d4ed8;">${ row.email }</small>
            </td>
            <td>${ row.telepon }</td>
            <td>${ row.alamat }</td>
            <td>
```

presentation/assets/js/app.js (Baris 46–90: Fungsi fetchCustomers GET & Render Tabel)

```
                <button type="button" class="btn btn-edit" onclick="isiFormEdit(${row.id})
    ">Edit</button>
                <button type="button" class="btn btn-hapus" onclick="hapusPelanggan(${row.
    id}, '${row.nama}')">Hapus</button>
            </td>
        </tr>
    `).join('');
}

// 2. Fungsi saat form Tambah / Edit disubmit (POST atau PUT dengan JSON)
document.getElementById('form-pelanggan').addEventListener('submit', async function (e) {
    e.preventDefault();
    sembunyikanPesan();

    const idEdit = document.getElementById('id_edit').value;
    const dataKirim = {
        nama: document.getElementById('nama').value,
        email: document.getElementById('email').value,
        telepon: document.getElementById('telepon').value,
        alamat: document.getElementById('alamat').value
    };

    // Jika id_edit terisi gunakan method PUT (Edit), jika kosong gunakan POST (Tambah)
    const method = idEdit !== '' ? 'PUT' : 'POST';
    const url = idEdit !== '' ? `${URL_API}?id=${idEdit}` : URL_API;

    const response = await fetch(url, {
        method: method,
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(dataKirim)
    });

    const hasil = await response.json();
    tampilkanLogJson(`${method} ${url} (HTTP ${response.status})`, hasil);

    if (hasil.success) {
        tampilkanSukses(hasil.message);
        resetForm();
        muatDataPelanggan(document.getElementById('input-cari').value);
    } else {
        tampilkanError(hasil.errors || [hasil.message]);
    }
});

// 3. Fungsi mengambil 1 data pelanggan ke dalam form untuk diedit (GET ?id=...)
async function isiFormEdit(id) {
    sembunyikanPesan();
```

Luaran 2 (File 3/7 — Presentation Layer Bagian 2/2): app.js

File: 3-layered-app/presentation/assets/js/app.js (Baris 91 s/d 176 Selesai) — Fetch POST, PUT, DELETE & Event UI

Presentation Layer (File 3 Lanjutan): Mengirim data form sebagai JSON (JSON.stringify) via metode POST/PUT/DELETE tanpa me-reload seluruh halaman.

presentation/assets/js/app.js (Baris 91–134: Submit Form POST/PUT JSON & Edit Mode)

```
091      const url = `${URL_API}?id=${id}`;
092      const response = await fetch(url);
093      const hasil = await response.json();
094
095      tampilkanLogJson(`GET ${url} (HTTP ${response.status})`, hasil);
096
097      if (hasil.success && hasil.data) {
098          document.getElementById('id_edit').value = hasil.data.id;
099          document.getElementById('nama').value = hasil.data.nama;
100          document.getElementById('email').value = hasil.data.email;
101          document.getElementById('telepon').value = hasil.data.telepon;
102          document.getElementById('alamat').value = hasil.data.alamat;
103
104          document.getElementById('judul-form').textContent = 'Edit Pelanggan #' + hasil.dat
     a.id;
105          document.getElementById('tombol-simpan').textContent = 'Simpan Perubahan';
106          document.getElementById('tombol-batal').style.display = 'inline-block';
107      }
108  }
109
110  // 4. Fungsi menghapus pelanggan berdasarkan ID (DELETE ?id=...)
111  async function hapusPelanggan(id, nama) {
112      if (!confirm(`Hapus pelanggan "${nama}"?`)) return;
113      sembunyikanPesan();
114
115      const url = `${URL_API}?id=${id}`;
116      const response = await fetch(url, { method: 'DELETE' });
117      const hasil = await response.json();
118
119      tampilkanLogJson(`DELETE ${url} (HTTP ${response.status})`, hasil);
120
121      if (hasil.success) {
122          tampilkanSukses(hasil.message);
123          resetForm();
124          muatDataPelanggan(document.getElementById('input-cari').value);
125      } else {
126          tampilkanError(hasil.errors || [hasil.message]);
127      }
128  }
129
130  // 5. Fungsi pencarian berdasarkan nama atau email
131  document.getElementById('form-cari').addEventListener('submit', function (e) {
132      e.preventDefault();
133      const kataKunci = document.getElementById('input-cari').value.trim();
134      muatDataPelanggan(kataKunci);
```

presentation/assets/js/app.js (Baris 135–176 Selesai: Hapus Pelanggan DELETE & Search)

```
135  });
136
137  function resetPencarian() {
138      document.getElementById('input-cari').value = '';
139      muatDataPelanggan('');
140  }
141
142  // Fungsi bantu mereset form kembali ke mode Tambah
143  function resetForm() {
144      document.getElementById('form-pelanggan').reset();
145      document.getElementById('id_edit').value = '';
146      document.getElementById('judul-form').textContent = 'Tambah Pelanggan Baru';
147      document.getElementById('tombol-simpan').textContent = '+ Tambah Pelanggan';
148      document.getElementById('tombol-batal').style.display = 'none';
149  }
150
151  // Fungsi bantu menampilkan pesan sukses / error
152  function tampilkanSukses(pesan) {
153      const box = document.getElementById('box-sukses');
154      box.textContent = '⊡ ' + pesan;
155      box.style.display = 'block';
156  }
157
158  function tampilkanError(daftarError) {
159      const box = document.getElementById('box-error');
160      const ul = document.getElementById('list-error');
161      ul.innerHTML = daftarError.map(err => `<li>${err}</li>`).join('');
162      box.style.display = 'block';
163  }
164
165  function sembunyikanPesan() {
166      document.getElementById('box-sukses').style.display = 'none';
167      document.getElementById('box-error').style.display = 'none';
168  }
169
170  function tampilkanLogJson(infoMethod, dataJson) {
171      document.getElementById('info-endpoint').textContent = infoMethod;
172      document.getElementById('isi-json').textContent = JSON.stringify(dataJson, null, 2);
173  }
174
175  // Panggil otomatis saat halaman pertama kali dibuka
176  muatDataPelanggan();
```

Luaran 2 (File 4/7 — Business Logic Layer): api/customers.php

File: 3-layered-app/business-logic/api/customers.php (Baris 1 s/d 64 Lengkap) — Endpoint REST API & Pertukaran JSON

Business Logic Layer (File 4): Menerima HTTP GET, POST, PUT, DELETE dari Frontend, memanggil CustomerService, dan mengirim output JSON (json_encode).

business-logic/api/customers.php (Baris 1–33: Header JSON, Dependency, & Handler GET)

```php
<?php
/**
 * ============================================================================
 * LAYER 2: BUSINESS LOGIC LAYER — REST API Endpoint (api/customers.php)
 * Tugas : Menerima request HTTP (GET, POST, PUT, DELETE) dari Presentation
 *         Layer (Frontend), memprosesnya melalui CustomerService, dan
 *         mengirimkan hasil kembali dalam format JSON.
 * ============================================================================
 */

header('Content-Type: application/json; charset=utf-8');

require_once dirname(__DIR__) . '/CustomerService.php';

try {
    $service = new CustomerService();
    $method  = $_SERVER['REQUEST_METHOD'];

    // Membaca data JSON yang dikirim oleh Frontend (untuk POST dan PUT)
    $rawBody = file_get_contents('php://input');
    $input   = json_decode($rawBody, true);
    if (!is_array($input)) {
        $input = $_POST;
    }

    $id = isset($_GET['id']) ? (int) $_GET['id'] : 0;

    if ($method === 'GET') {
        // Jika ada parameter ?id=... ambil 1 pelanggan, jika tidak ambil semua / cari (?q
  =...)
        if ($id > 0) {
            $hasil = $service->getCustomerById($id);
        } else {
            $keyword = $_GET['q'] ?? '';
```

business-logic/api/customers.php (Baris 34–64 Selesai: Handler POST, PUT, DELETE JSON)

```php
            $hasil   = $service->listCustomers($keyword);
        }
    } elseif ($method === 'POST') {
        // Tambah pelanggan baru
        $hasil = $service->createCustomer($input);
    } elseif ($method === 'PUT') {
        // Edit pelanggan berdasarkan ID
        $hasil = $service->updateCustomer($id, $input);
    } elseif ($method === 'DELETE') {
        // Hapus pelanggan berdasarkan ID
        $hasil = $service->deleteCustomer($id);
    } else {
        $hasil = [
            'status_code' => 405,
            'success'     => false,
            'message'     => 'Method HTTP tidak didukung.',
        ];
    }

    // Kirim status HTTP dan output JSON ke Presentation Layer
    http_response_code($hasil['status_code'] ?? 200);
    echo json_encode($hasil, JSON_PRETTY_PRINT | JSON_UNESCAPED_UNICODE);
} catch (Throwable $e) {
    http_response_code(500);
    echo json_encode([
        'status_code' => 500,
        'success'     => false,
        'message'     => $e->getMessage(),
        'errors'      => [$e->getMessage()],
    ], JSON_PRETTY_PRINT | JSON_UNESCAPED_UNICODE);
}
```

Luaran 2 (File 5/7 — Business Logic Layer Bagian 1/2): CustomerService.php

File: 3-layered-app/business-logic/CustomerService.php (Baris 1 s/d 100 dari 197 Baris) — Aturan Bisnis & Validasi

Business Logic Layer (File 5): Menangani proses bisnis utama aplikasi, mengambil data via Repository, serta memvalidasi kelengkapan & keunikan email.

business-logic/CustomerService.php (Baris 1–50: Constructor & getAllCustomers)

```php
<?php
/**
 * ============================================================================
 * LAYER 2: BUSINESS LOGIC LAYER — CustomerService.php
 * Tugas : Menangani aturan bisnis & validasi aplikasi Customer Management:
 *         1. Memeriksa kelengkapan input (nama, email, telepon, alamat)
 *         2. Memeriksa format email yang benar
 *         3. Memeriksa keunikan email (meminta bantuan Data Layer)
 *         4. Mengatur proses Tampil, Tambah, Edit, Hapus, dan Pencarian
 *
 * Catatan : Layer ini TIDAK menulis query SQL dan TIDAK membuat kode HTML.
 * ============================================================================
 */

require_once dirname(__DIR__) . '/data-access/CustomerRepository.php';

class CustomerService
{
    private $repo;

    public function __construct()
    {
        $this->repo = new CustomerRepository();
    }

    // 1. Ambil daftar pelanggan (Semua atau hasil Pencarian Nama/Email)
    public function listCustomers($keyword = '')
    {
        $keyword = trim((string) $keyword);
        $data = ($keyword !== '')
            ? $this->repo->search($keyword)
            : $this->repo->findAll();

        return [
            'status_code' => 200,
            'success'     => true,
            'message'     => 'Data pelanggan berhasil diambil.',
            'meta'        => [
                'total_all'      => $this->repo->countAll(),
                'total_filtered' => count($data),
                'keyword'        => $keyword,
            ],
            'data'        => $data,
        ];
    }

    // 2. Ambil 1 data pelanggan berdasarkan ID
    public function getCustomerById($id)
    {
        $pelanggan = $this->repo->findById((int) $id);
```

business-logic/CustomerService.php (Baris 51–100: getCustomerById & createCustomer)

```php
        if (!$pelanggan) {
            return [
                'status_code' => 404,
                'success'     => false,
                'message'     => "Pelanggan dengan ID #$id tidak ditemukan.",
                'errors'      => ["Pelanggan dengan ID #$id tidak ditemukan."],
            ];
        }

        return [
            'status_code' => 200,
            'success'     => true,
            'message'     => 'Detail pelanggan ditemukan.',
            'data'        => $pelanggan,
        ];
    }

    // 3. Tambah pelanggan baru (dengan Validasi Kelengkapan & Keunikan Email)
    public function createCustomer($input)
    {
        $data   = $this->rapikanInput($input);
        $errors = $this->validasiData($data, null);

        // Jika validasi gagal, tolak penyimpanan & kirim daftar error
        if (!empty($errors)) {
            return [
                'status_code' => 422,
                'success'     => false,
                'message'     => 'Validasi data pelanggan gagal.',
                'errors'      => $errors,
            ];
        }

        // Jika validasi lolos, minta Data Layer menyimpan ke database
        $pelangganBaru = $this->repo->create($data);

        return [
            'status_code' => 201,
            'success'     => true,
            'message'     => "Pelanggan baru '{$data['nama']}' berhasil ditambahkan!",
            'data'        => $pelangganBaru,
        ];
    }

    // 4. Edit pelanggan (dengan Validasi Kelengkapan & Keunikan Email)
    public function updateCustomer($id, $input)
    {
        $id = (int) $id;
        $lama = $this->repo->findById($id);
        if (!$lama) {
```

Luaran 2 (File 5/7 — Business Logic Layer Bagian 2/2): CustomerService.php

File: 3-layered-app/business-logic/CustomerService.php (Baris 101 s/d 197 Selesai) — Update, Delete, & Fungsi Validasi

Business Logic Layer (File 5 Lanjutan): Memeriksa kelengkapan kolom wajib, format email, dan memastikan tidak ada email ganda (HTTP 422 Unprocessable Entity).

business-logic/CustomerService.php (Baris 101–150: updateCustomer & deleteCustomer)

```
            return [
                'status_code' => 404,
                'success'     => false,
                'message'     => "Pelanggan dengan ID #$id tidak ditemukan.",
                'errors'      => ["Data pelanggan tidak ditemukan."],
            ];
        }

        $data   = $this->rapikanInput($input);
        $errors = $this->validasiData($data, $id);

        if (!empty($errors)) {
            return [
                'status_code' => 422,
                'success'     => false,
                'message'     => 'Validasi perubahan data gagal.',
                'errors'      => $errors,
            ];
        }

        $pelangganUpdate = $this->repo->update($id, $data);

        return [
            'status_code' => 200,
            'success'     => true,
            'message'     => "Data pelanggan '{$data['nama']}' berhasil diperbarui!",
            'data'        => $pelangganUpdate,
        ];
    }

    // 5. Hapus pelanggan berdasarkan ID
    public function deleteCustomer($id)
    {
        $id = (int) $id;
        $lama = $this->repo->findById($id);
        if (!$lama) {
            return [
                'status_code' => 404,
                'success'     => false,
                'message'     => "Pelanggan dengan ID #$id tidak ditemukan.",
                'errors'      => ["Data pelanggan tidak ditemukan."],
            ];
        }

        $this->repo->delete($id);

        return [
            'status_code' => 200,
            'success'     => true,
            'message'     => "Data pelanggan '{$lama['nama']}' berhasil dihapus.",
```

business-logic/CustomerService.php (Baris 151–197 Selesai: Fungsi validateCustomerInput)

```
            'data'        => ['deleted_id' => $id],
        ];
    }

    // Fungsi bantu: merapikan spasi dan huruf kecil pada email
    private function rapikanInput($input)
    {
        return [
            'nama'    => trim((string) ($input['nama'] ?? '')),
            'email'   => strtolower(trim((string) ($input['email'] ?? ''))),
            'telepon' => trim((string) ($input['telepon'] ?? '')),
            'alamat'  => trim((string) ($input['alamat'] ?? '')),
        ];
    }

    // Fungsi utama aturan bisnis: Validasi Kelengkapan & Keunikan Email
    private function validasiData($data, $excludeId = null)
    {
        $errors = [];

        // A. Cek kelengkapan seluruh kolom wajib
        if ($data['nama'] === '') {
            $errors[] = 'Nama pelanggan wajib diisi.';
        }
        if ($data['email'] === '') {
            $errors[] = 'Alamat email wajib diisi.';
        } elseif (!filter_var($data['email'], FILTER_VALIDATE_EMAIL)) {
            $errors[] = 'Format alamat email tidak valid.';
        }
        if ($data['telepon'] === '') {
            $errors[] = 'Nomor telepon wajib diisi.';
        }
        if ($data['alamat'] === '') {
            $errors[] = 'Alamat pelanggan wajib diisi.';
        }

        // B. Cek keunikan email melalui Data Layer
        if ($data['email'] !== '' && filter_var($data['email'], FILTER_VALIDATE_EMAIL)) {
            $duplikat = $this->repo->findByEmail($data['email'], $excludeId);
            if ($duplikat) {
                $errors[] = "Email '{$data['email']}' sudah digunakan oleh pelanggan '{$du
plikat['nama']}'. Gunakan email yang unik!";
            }
        }

        return $errors;
    }
}
```

Luaran 2 (File 6/7 — Data Layer): data-access/Database.php

File: 3-layered-app/data-access/Database.php (Baris 1 s/d 49 Lengkap) — Koneksi PDO MySQL & Auto-Seed Tabel

Data Layer (File 6): Mengisolasi konfigurasi koneksi database MySQL (PDO) di satu tempat sehingga tidak tersebar di file tampilan maupun logika bisnis.

data-access/Database.php (Baris 1–25: Class Database & Konfigurasi Host/DB MySQL)

```php
<?php
/**
 * ============================================================================
 * LAYER 3: DATA LAYER — Database.php
 * Tugas : Membuka koneksi langsung ke database MySQL (customer_management).
 *         Struktur tabel dan data awal bersumber dari file SQL:
 *         database/customer_management.sql
 * ============================================================================
 */

class Database
{
    private static $pdo = null;

    public static function getConnection()
    {
        if (self::$pdo === null) {
            $host     = "localhost";
            $dbname   = "customer_management";
            $username = "root";
            $password = "";

            $options = [
                PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
                PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
```

data-access/Database.php (Baris 26–49 Selesai: Koneksi PDO & Auto-Import SQL)

```php
            ];

            try {
                // Langsung konek ke database customer_management di MySQL
                self::$pdo = new PDO("mysql:host=$host;dbname=$dbname;charset=utf8mb4", $u
sername, $password, $options);
            } catch (PDOException $e) {
                // Jika database customer_management belum di-import di MySQL,
                // otomatis jalankan file database/customer_management.sql
                $sqlFile = dirname(__DIR__, 2) . "/database/customer_management.sql";
                if (strpos($e->getMessage(), "1049") !== false && file_exists($sqlFile)) {
                    $pdoSetup = new PDO("mysql:host=$host;charset=utf8mb4", $username, $pa
ssword, $options);
                    $pdoSetup->exec(file_get_contents($sqlFile));
                    self::$pdo = new PDO("mysql:host=$host;dbname=$dbname;charset=utf8mb4"
, $username, $password, $options);
                } else {
                    throw new RuntimeException(
                        "Koneksi MySQL gagal. Pastikan MySQL di XAMPP/Laragon sudah dinyal
akan. Detail: " . $e->getMessage()
                    );
                }
            }
        }

        return self::$pdo;
    }
}
```

Luaran 2 (File 7/7 — Data Layer): data-access/CustomerRepository.php

File: 3-layered-app/data-access/CustomerRepository.php (Baris 1 s/d 101 Lengkap) — Query SQL Prepared Statements

Data Layer (File 7): Berisi seluruh query SQL (SELECT, pencarian LIKE, cek email unik, INSERT, UPDATE, DELETE) menggunakan PDO Prepared Statements.

data-access/CustomerRepository.php (Baris 1–52: findAll, Search, findById, emailExists)

```php
<?php
/**
 * ============================================================================
 * LAYER 3: DATA LAYER — CustomerRepository.php
 * Tugas : Menjalankan query SQL (SELECT, INSERT, UPDATE, DELETE, Pencarian)
 *         ke tabel `pelanggan`.
 *         Dipanggil oleh Business Logic Layer (CustomerService.php).
 * ============================================================================
 */

require_once __DIR__ . '/Database.php';

class CustomerRepository
{
    private $db;

    public function __construct()
    {
        $this->db = Database::getConnection();
    }

    // 1. Mengambil seluruh data pelanggan (Fitur Tampil Pelanggan)
    public function findAll()
    {
        $stmt = $this->db->query("SELECT * FROM pelanggan ORDER BY id DESC");
        return $stmt->fetchAll();
    }

    // 2. Mencari pelanggan berdasarkan Nama atau Email (Fitur Pencarian)
    public function search($keyword)
    {
        $stmt = $this->db->prepare("SELECT * FROM pelanggan WHERE nama LIKE ? OR email LIK
E ? ORDER BY id DESC");
        $stmt->execute(["%$keyword%", "%$keyword%"]);
        return $stmt->fetchAll();
    }

    // 3. Mengambil 1 pelanggan berdasarkan ID (Untuk Edit / Cek Hapus)
    public function findById($id)
    {
        $stmt = $this->db->prepare("SELECT * FROM pelanggan WHERE id = ?");
        $stmt->execute([(int) $id]);
        $row = $stmt->fetch();
        return $row ?: null;
    }

    // 4. Mencari pelanggan berdasarkan Email (Untuk Validasi Keunikan Email)
    public function findByEmail($email, $excludeId = null)
    {
        if ($excludeId !== null) {
            $stmt = $this->db->prepare("SELECT * FROM pelanggan WHERE LOWER(email) = LOWER
(?) AND id != ?");
            $stmt->execute([$email, (int) $excludeId]);
        } else {
```

data-access/CustomerRepository.php (Baris 53–101 Selesai: Query INSERT, UPDATE, DELETE)

```php
            $stmt = $this->db->prepare("SELECT * FROM pelanggan WHERE LOWER(email) = LOWER
(?)");
            $stmt->execute([$email]);
        }
        $row = $stmt->fetch();
        return $row ?: null;
    }

    // 5. Menyimpan pelanggan baru ke database (Fitur Tambah Pelanggan)
    public function create($data)
    {
        $stmt = $this->db->prepare("INSERT INTO pelanggan (nama, email, telepon, alamat) V
ALUES (?, ?, ?, ?)");
        $stmt->execute([
            $data['nama'],
            $data['email'],
            $data['telepon'],
            $data['alamat'],
        ]);
        $idBaru = (int) $this->db->lastInsertId();
        return $this->findById($idBaru);
    }

    // 6. Mengubah data pelanggan di database (Fitur Edit Pelanggan)
    public function update($id, $data)
    {
        $stmt = $this->db->prepare("UPDATE pelanggan SET nama = ?, email = ?, telepon = ?,
 alamat = ? WHERE id = ?");
        $stmt->execute([
            $data['nama'],
            $data['email'],
            $data['telepon'],
            $data['alamat'],
            (int) $id,
        ]);
        return $this->findById((int) $id);
    }

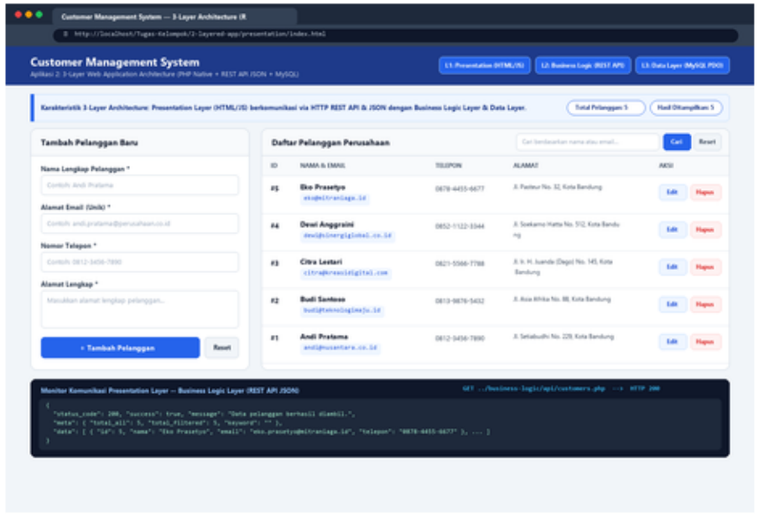
    // 7. Menghapus data pelanggan dari database (Fitur Hapus Pelanggan)
    public function delete($id)
    {
        $stmt = $this->db->prepare("DELETE FROM pelanggan WHERE id = ?");
        $stmt->execute([(int) $id]);
        return $stmt->rowCount() > 0;
    }

    // Menghitung total seluruh pelanggan
    public function countAll()
    {
        return (int) $this->db->query("SELECT COUNT(*) FROM pelanggan")->fetchColumn();
    }
}
```

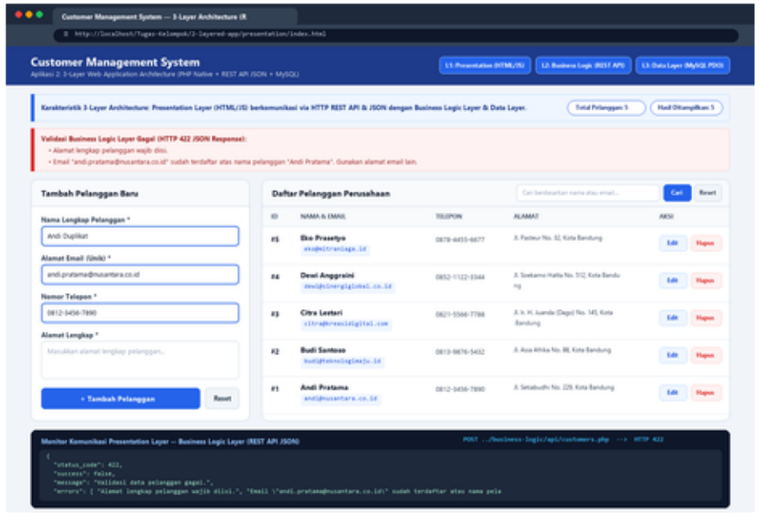
Luaran 5 (D): Screenshot Tampilan Web Aplikasi 2 (3-Layer REST API)

Lengkap 3 Skenario Demo pada 3-layered-app/: Dashboard JSON 200 OK, Validasi 422 JSON, & Pencarian REST API

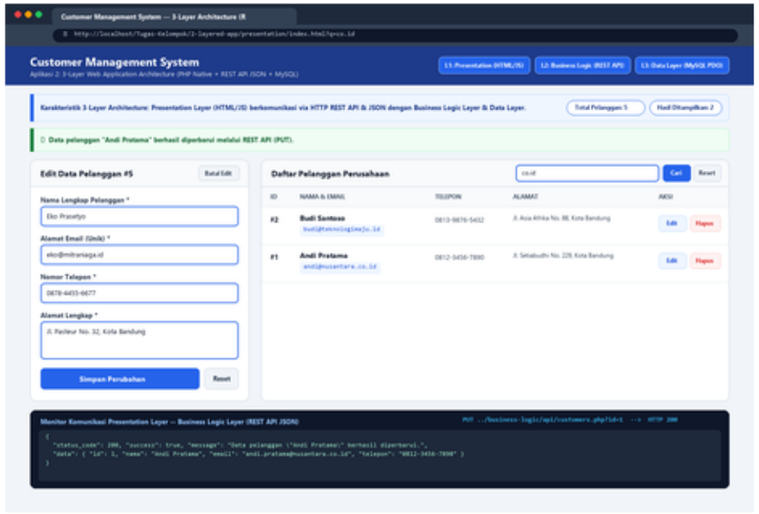
1. Dashboard 3-Layer + Monitor JSON (HTTP 200 OK)



2. Validasi Business Logic (HTTP 422 JSON)



3. Edit & Pencarian Asinkron via Fetch API (?search=...)



Luaran 4 (C): Perbandingan Alur Proses Request & Response

Ringkasan Cara Kerja di Balik Layar Setelah Demo Kedua Aplikasi (6 Tahap vs 7 Tahap REST API)

PERBANDINGAN ALUR EKSEKUSI: NON-LAYERED vs 3-LAYER ARCHITECTURE

Studi Kasus: Proses Tambah Pelanggan & Validasi Keunikan Email pada Customer Management System

PENDEKATAN 1: NON-LAYERED ARCHITECTURE (MONOLITIK)

Tahap 1 — Input Form HTML

Pengguna mengisi form di index.php lalu menekan tombol Submit.

Tahap 2 — HTTP POST Form-Urlencoded

Browser mengirim form ke index.php (memicu pemuatan ulang halaman).

Tahap 3 — Validasi Prosedural di index.php

Blok if ($_POST) memeriksa field kosong & menjalankan query cek email.

Tahap 4 — Eksekusi INSERT Langsung

File index.php langsung mengeksekusi query INSERT INTO pelanggan.

Tahap 5 — Query Ulang & Render HTML Utuh

File index.php menjalankan SELECT * lalu mengembalikan seluruh HTML.

PENDEKATAN 2: 3-LAYER ARCHITECTURE (REST API & JSON)

Tahap 1 — Presentation Layer (app.js)

Menangkap submit form & mengirim HTTP POST JSON via fetch() async.

Tahap 2 — REST API Controller (api/customers.php)

Menerima JSON payload & meneruskannya ke CustomerService.

Tahap 3 — Business Logic Layer (CustomerService.php)

Memvalidasi kelengkapan data, format email, & memanggil cek duplikasi.

Tahap 4 — Data Layer (CustomerRepository.php)

Menjalankan Prepared Statement findByEmail() & create() ke MySQL.

Tahap 5 — JSON Response (HTTP 201 / 422)

Backend mengirim JSON; Presentation Layer memperbarui tabel DOM.

Luaran 6: Tabel Hasil Pengujian & Perbandingan Akhir

Perbandingan Unsur Uji Coba, Tanpa Layer vs 3-Layer, Siapa yang Lebih Unggul, beserta Alasannya

Unsur yang Diuji Coba	Tanpa Layer (Non-Layered)	Menggunakan 3-Layer	Lebih Unggul	Alasan Utama
1. Pemisahan File/Folder	Tanpa batas layer (di sini 1 file)	Terpisah tegas di 3 folder layer	**3-Layer**	**Kode rapi & tanggung jawab tiap layer jelas**
2. Kecepatan Buat Awal	Sangat cepat (langsung menyatu)	Lebih lama (buat API & 3 layer)	**Non-Layered**	**Praktis untuk aplikasi kecil/sederhana**
3. Individu vs Kerja Tim	Cocok individu, sulit bagi tim	Bisa dibagi paralel ke 3 orang/tim	**3-Layer**	**Tiap anggota kerjakan folder berbeda tanpa bentrok**
4. Penelusuran Error (Bug)	Sulit dicari karena kode campur	Mudah dilacak sesuai foldernya	**3-Layer**	**Langsung tahu letak bug (UI / Logic / SQL)**
5. Kecepatan Respons UI	Lambat (Full Page Reload)	Cepat (Fetch JSON tanpa reload)	**3-Layer**	**Hanya bertukar data JSON yang ringan**
6. Keamanan Edit Desain	Rawan merusak kode PHP/SQL	Aman (HTML/CSS terpisah penuh)	**3-Layer**	**Ubah tampilan tidak menyentuh kode database**
7. Penambahan Fitur Baru	Kode makin panjang & bercampur	Tinggal tambah fungsi per layer	**3-Layer**	**Mudah dikembangkan tanpa merusak kode lama**
8. Reusabilitas (Mobile)	Tidak bisa (output berupa HTML)	Bisa dipakai Web & Android/iOS	**3-Layer**	**REST API menghasilkan format standar JSON**

Luaran 7: Jawaban 5 Pertanyaan Diskusi (Halaman 17 PDF)

Analisis Perbandingan Berdasarkan Pengalaman Membangun Kedua Aplikasi

1. Mana yang lebih mudah dibuat?

Non-Layered lebih cepat dibuat di awal untuk aplikasi kecil/individu karena seluruh kode cukup ditulis langsung tanpa membuat layer terpisah.

2. Mana yang lebih mudah diperbaiki saat terjadi kesalahan (bug)?

3-Layer jauh lebih mudah ditelusuri karena bug langsung terisolasi sesuai foldernya (apakah di presentation/, business-logic/, atau data-access/).

3. Mana yang lebih mudah dikembangkan saat fitur bertambah?

3-Layer jauh lebih unggul karena mendukung kerja tim secara paralel dan endpoint REST API JSON-nya bisa dipakai ulang oleh aplikasi lain.

4. Apa risiko jika seluruh kode ditempatkan dalam satu file?

Kode tercampur aduk (spaghetti code), edit desain HTML rawan merusak query SQL, rawan bentrok saat kerja tim, dan sulit melacak bug.

5. Apakah aplikasi tanpa 3-layer selalu lebih buruk?

Tidak selalu. Untuk tugas individu atau web kecil sederhana, Non-Layered lebih cepat. Namun untuk sistem bisnis & kerja tim, 3-Layer mutlak lebih unggul.

Ringkasan 8 Luaran Wajib (Hal. 22 PDF) & Kesimpulan Akhir

Seluruh Luaran Telah Tercantum Lengkap di Dalam Presentasi Ini, Laporan PDF, & Repositori Proyek

Checklist 8 Luaran Wajib (Halaman 22 PDF)

[OK] 1. Source Code Non-Layered	: Lengkap 310 baris di Slide 9–11 & Bab 5.1 Laporan
[OK] 2. Source Code 3-Layer (7 File)	: Lengkap seluruh 7 file di Slide 15–24 & Bab 5.2 Laporan
[OK] 3. File Database SQL	: Lengkap 28 baris di Slide 6 & Bab 3.3 Laporan
[OK] 4. Diagram Arsitektur	: Lengkap 3 diagram di Slide 7, 13, 26 & Bab 4 Laporan
[OK] 5. Screenshot Aplikasi	: Lengkap 2 Screenshot Code + 6 Screenshot Web (Slide 8,12,14,25)
[OK] 6. Tabel Hasil Pengujian	: Lengkap 8 unsur perbandingan di Slide 27 & Bab 7 Laporan
[OK] 7. Laporan Analisis (PDF)	: Lengkap 8 Bab + seluruh kode sumber di folder laporan/
[OK] 8. Slide Presentasi Kelompok	: Lengkap 29 slide (PPTX & PDF) di folder presentasi/

Kesimpulan Akhir Perbandingan Arsitektur

Non-Layered Architecture Unggul pada:

- Kecepatan pembuatan awal untuk aplikasi sangat kecil atau skrip sederhana yang dikerjakan 1 orang (individu).

3-Layer Architecture Unggul Mutlak pada (7 dari 8 Unsur):

- Pemisahan file/folder sangat rapi (presentation, logic, data).
- Sangat ideal untuk KERJA TIM secara paralel tanpa bentrok.
- Penelusuran kesalahan (debugging) sangat cepat & terarah.
- Respons UI lebih cepat (Fetch JSON tanpa reload halaman).
- Aman mengubah desain HTML/CSS tanpa menyentuh SQL.
- Mudah dikembangkan & REST API JSON siap dipakai Mobile.

TERIMA KASIH — SESI TANYA JAWAB & DISKUSI

Seluruh 8 Luaran Wajib (Source Code Lengkap, SQL, Diagram, Screenshot, & Tabel Pengujian) Tercantum di PPT & Laporan.